Micro Frontends in Practice: The Complete Guide

Building Scalable Web Applications with Micro Frontend Architecture

---

What is a Micro Frontend?

Just as **Microservices** broke the monolithic backend into independently deployable services, **Micro Frontends (MFE)** apply the same thinking to the browser.

Instead of one large React or Angular application owning the entire UI, you split the frontend into **smaller, independently deployable applications** — each owned by a different team, built in any framework, and deployed on its own schedule.

> Think of it as **"microservices for the browser"** — the same isolation, the same team autonomy, now applied to the frontend layer.

The Problem It Solves

Large frontend applications break in predictable ways. What starts as one team's product eventually carries dozens of modules, hundreds of screens, and backends that have long since evolved into independent microservices — yet the frontend is still one codebase, one build pipeline, one deployment.

The symptoms appear across two dimensions: what the **product cannot do**, and what the **teams cannot do**. Both need to be present before MFE is the justified answer.

Product Symptoms — What the Application Can No Longer Do

What you observe	What it tells you
A change in one module breaks unrelated features	No enforced boundary — DOM, state, and runtime are fully shared
Modules need different release cadences but must ship as one	Deployment is too coarse — fast and slow sections are artificially coupled
A legacy section cannot be rewritten without freezing everything else	The monolith owns all — there is no way to swap one area independently
A new framework cannot be introduced in just one area	The entire team must agree and migrate the whole codebase at once
Users download code for features they will never visit	No lazy-loading boundary — every module ships in every bundle

Team Symptoms — What the Teams Can No Longer Do

What you observe	What it tells you
One team's PR regularly breaks another team's work	Shared codebase — shared blast radius, no hard ownership boundary
CI takes 30+ minutes for a small isolated change	Every commit rebuilds and retests everything — no scoped build
A PR needs sign-off from three unrelated teams	Ownership is unclear — no one knows where the boundary is
Teams wait on each other for every release	Deployment is a shared event — one team can delay the whole platform

> **The clearest signal:** when your product team says "*we cannot ship X until Y team finishes Z*" — and X and Y have nothing to do with each other — the frontend has become the bottleneck.

Is MFE the Right Answer?

MFE is **not** the right answer for every situation. A well-structured monolith with clear internal module boundaries is simpler and equally effective for small, single-team products.

MFE is justified when **at least two** of these apply:

- **Multiple teams** own different sections and need to deploy independently
- **Different release cadences** — compliance-gated and fast-moving sections cannot share a pipeline
- **Framework diversity** — an acquired app, a vendor product, or a legacy section cannot be migrated
- **Scale** — the product has grown past the point where one team can understand the whole codebase
- **Compliance isolation** — regulations require an independent audit trail for a specific section

The right question is not "should we use MFE?" — it is "do we have boundaries that need to be independently deployable?"

- **Yes** → MFE gives each boundary its own build, its own deployment, its own team ownership
- **No** → A modular monolith with enforced internal boundaries is simpler and equally maintainable

---

Choosing Your Strategy — Monorepo, Polyrepo, or Hybrid

Once MFE is justified, the most consequential decision is not which framework to use — it is how your MFEs relate to each other in terms of repositories, ownership, and deployment pipelines. This decision is driven by your **organisational structure**, not technical preference.

If your primary problem is...	Strategy	What it gives you
Internal teams blocked on each other inside one repo	**Monorepo (Nx)**	Nx enforces ownership boundaries, `nx affected` keeps CI fast
An acquired company or vendor product to integrate	**Polyrepo**	External team keeps their own repo — Shell loads their MFE by URL only
Compliance-regulated sections (payments, health data)	**Polyrepo**	Isolated pipeline, isolated audit trail, no shared deployment lifecycle
Mix of internal teams and regulated/external sections	**Hybrid**	Core in monorepo, regulated or external MFEs in their own repos
Shared design system + independent product deployments	**Hybrid**	Shared libs as versioned npm packages, each MFE installs and deploys freely

How to decide:

- Are all your teams internal and on the same CI platform? → **Monorepo with Nx**
- Do you have external vendors, acquired apps, or strict compliance isolation? → **Polyrepo** for those
- Do you have both? → **Hybrid** — monorepo core, polyrepo for isolated concerns

> **The most common mistake:** choosing Polyrepo because it feels more autonomous — then spending months managing version drift across 15 repos. If all teams are internal, Monorepo with Nx is almost always the faster, safer path.

	Monorepo	Polyrepo	Hybrid
Best for	Internal teams, shared domain	External vendors, compliance isolation	Mixed — internal core + regulated parts
Shared lib updates	Atomic — one PR covers lib and all consumers	Manual version bump per repo	Versioned npm — teams upgrade on their schedule
Release independence	Coordinated, but `nx affected` scopes builds	Fully independent	MFEs independent, shared libs versioned
Compliance isolation	Shared policies across all	Full isolation per repo	Per-domain isolation

What MFE Solves — Problem to Solution

Problem	How MFE solves it
One deploy for the entire frontend	Each MFE has its own CI/CD — deploy only what changed
Framework lock-in	Each MFE can use any framework — React, Angular, Vue, or plain JS
Slow builds	Each MFE builds independently — Nx `affected` runs only what changed
Blocked cross-team releases	Teams deploy on their own schedule
Cannot integrate partner or vendor UI	Partner hosts their MFE — Shell loads it by URL, no code access required
Compliance isolation	Regulated MFE runs in its own isolated pipeline — independent audit trail
Legacy UI that cannot be replaced safely	Replace one MFE at a time — the Shell stays stable while sections are rewritten
Giant bundle on every page	Shell lazy-loads only the MFE for the current route

---

Core Concepts

1. Shell (Host Application)

The Shell is the container that users first load in their browser. Its responsibilities:

- Top-level layout — navigation, header, footer
- Authentication and routing
- Loading each Micro Frontend on demand

The Shell does **not** contain business logic. It is purely an orchestrator. Every MFE it loads could be replaced without touching the Shell code.

2. Remote Applications (MFEs)

Each MFE is an independent application that:

- Has its own build and deployment pipeline
- Exposes a URL endpoint (`remoteEntry.js`) that the Shell loads at runtime
- Can be built in any framework — React, Angular, Vue, or plain JavaScript
- Lives in its own repo (Polyrepo) or in a monorepo alongside the Shell (Monorepo)

In this project:

- **Shell + React MFE + Vue MFE + Angular MFE** → `github.com/sudeep31/MFEDemo` (Monorepo)
- **Angular Todo MFE** → `github.com/sudeep31/angularTodo` (Polyrepo — completely separate repo)

3. Shared Libraries

Some code must be consistent across all MFEs:

- **Design System** — Bootstrap 5 SCSS, shared design tokens
- **Web Components** — framework-agnostic UI elements (buttons, headers, modals)
- **Auth Library** — shared token store so users never log in twice

---

How It Works — The Connection

The Shell reads a manifest file at startup — a simple JSON registry of all MFEs. When a user navigates to a route, the Shell fetches that MFE's JavaScript bundle from its URL and mounts it.

```
User opens browser
    ↓
Shell loads — reads mfe-manifest.json
    ↓
{ "react-app":   "https://cdn.../remoteEntry.js",
  "angular-app": "https://cdn.../remoteEntry.js" }
    ↓
User navigates to /react-app
    ↓
Shell fetches remoteEntry.js — React MFE mounts inside Shell layout
    ↓
User sees the full page — seamlessly
```

The key insight: the Shell only knows a URL, not the code. Each MFE is loaded at runtime from wherever it is hosted — CDN, cloud storage, or a local dev server.

The Manifest — Central Registry of All MFEs

```json
{
  "reactApp": {
    "remoteEntry": "http://localhost:3001/remoteEntry.js",
    "exposedModule": "./App",
    "framework": "react",
    "route": "/react-app",
    "active": true
  },
  "angularTodo": {
    "remoteEntry": "http://localhost:4200/remoteEntry.js",
    "type": "native-federation",
    "webComponentTag": "angular-todo-app",
    "route": "/angular-todo",
    "active": true
  }
}
```

Each field has a clear job:

Field	Purpose
`remoteEntry`	URL of the MFE bundle — localhost in dev, CDN in production
`exposedModule`	Which component to load from the MFE
`framework`	Tells the Shell how to mount it — React direct or Web Component
`route`	URL path that activates this MFE
`active`	Feature flag — disable any MFE without redeployment

The Shell reads this file and generates navigation, route registration, and home page cards automatically. **Adding a new MFE requires no Shell code change** — only a new manifest entry.

Polyrepo Strategy — External MFEs by URL Only

In a Polyrepo setup, each MFE lives in its own Git repository, maintained by its own team:

```
MFEDemo repo     → Shell + React MFE + Vue MFE + Angular MFE
angularTodo repo → Standalone Angular Todo — completely separate
```

The Shell integrates the polyrepo MFE with one manifest entry:

```
"angularTodo": {
  "type": "native-federation",
  "remoteEntry": "http://localhost:4200/remoteEntry.js",
  "webComponentTag": "angular-todo-app",
  "route": "/angular-todo",
  "active": true
}
```

No Shell code was changed. No Shell rebuild was triggered. The Angular Todo team deploys independently — the Shell picks up their changes the next time a user loads the page.

Cross-Framework in Practice

One of the most powerful features of this architecture: the Shell does not care what framework an MFE uses.

MFE	Framework	How Shell loads it
React MFE	React 19	Direct lazy component via Webpack Module Federation
Vue MFE	Vue 3	Direct lazy component via Webpack Module Federation
Angular MFE	Angular 21 Zoneless	Web Component — `<mfe-angular>`
Angular Todo MFE	Angular 21 Polyrepo	Web Component — `<angular-todo-app>`

The **Web Component** pattern is the cross-framework bridge. Any framework can expose itself as a native HTML custom element. The Shell renders the HTML tag — it needs no knowledge of what runs inside.

Web Component Strategy — The Cross-Framework Bridge

Why Not Just Import Angular into React?

The React Shell uses Webpack Module Federation to load React and Vue MFEs directly — because they share the same runtime. Angular is a completely different runtime. You cannot `import()` an Angular component into a React tree.

Option	Problem
`import()` Angular directly	Angular's runtime and DI system do not work inside a React tree
iframe	No shared auth, no shared CSS, terrible UX, heavy overhead
single-spa lifecycle	Correct at scale — overkill for a POC with fewer than 10 MFEs
Web Component	Browser-native standard — React renders `<mfe-angular>` like any `<div>`

Web Components won because they are a **W3C standard**. No React adapter, no Angular adapter — just an HTML element that any host understands.

How Angular Registers as a Web Component

The key architectural decision is **how Angular bootstraps** as a Web Component:

```
// Standard Angular app — mounts to <app-root>, not reusable by a host
bootstrapApplication(App, appConfig);

// MFE approach — creates the Angular runtime WITHOUT mounting to the DOM
createApplication({ providers: [...] }).then((appRef) => {

  // Wrap the Angular component behind the Custom Elements v1 API
  const el = createCustomElement(TodoListComponent, { injector: appRef.injector });

  // Register the tag name in the browser's global custom element registry
  // Guard prevents double-registration if the script loads twice
  if (!customElements.get('angular-todo-app')) {
    customElements.define('angular-todo-app', el);
  }
});
```

Why this matters:

- `createApplication()` instead of `bootstrapApplication()` — Angular starts without claiming a DOM element. The host Shell controls where and when it appears.
- `createCustomElement()` — exposes the Angular component through the browser's standard Custom Elements API. The Shell sees an HTML element, not an Angular component.
- The guard on `customElements.get()` — if the Shell loads the script twice (e.g. hot-reload), it will not crash with a "already defined" error.

Why We Expose a Feature Component, Not the App Root

```
Wrong:  expose AppComponent
        AppComponent has <router-outlet>
        Angular Router activates → tries to own the URL
        Shell Router also owns the URL
        Two routers fight → broken navigation

Correct: expose TodoListComponent directly
         Self-contained feature component — no router
         Renders correctly inside any host
         Shell controls all routing
```

This applies to every Angular MFE that has routing: always expose a **feature component**, not the root app component.

Shell Loads It via Script Injection

```
// Shell — generic loader for any Angular Web Component MFE
// Driven entirely by mfe-manifest.json — no MFE-specific code in the Shell

function NativeFederationMfe({ config }) {
  useEffect(() => {
    // Step 1 — inject the script tag pointing to the MFE's bundle URL
    const script = document.createElement('script');
    script.src = config.remoteEntry;

    script.onload = () => {
      // Step 2 — once the script loads, the custom element is registered
      // The Shell just renders the HTML tag — Angular handles everything inside
      setStatus('ready');
    };

    document.head.appendChild(script);
  }, [config.remoteEntry]);

  // Step 3 — render the custom element tag from the manifest
  return <div ref={containerRef} />;  // config.webComponentTag renders here
}
```

No Angular-specific code in the Shell. Adding a third Angular MFE requires zero Shell code changes — just a new entry in `mfe.manifest.json`.

Pros and Cons of the Web Component Approach

Advantages:

Advantage	Detail
True framework independence	Shell renders an HTML tag — no knowledge of Angular
Browser-native standard	No polyfill needed in modern browsers — W3C spec
Perfect for polyrepos	Remote team publishes a URL, Shell team adds one manifest entry
Independent lifecycle	Angular manages its own change detection and DI — separate from React
Zero shared protocol	Works even when Shell and MFE use completely different federation tools

Disadvantages:

Disadvantage	When it matters
Attributes are strings only	Cannot pass complex React objects as HTML attributes — use DOM properties
Angular router conflict	Must expose feature components, not the App root
Double runtime in memory	React and Angular both load in the same tab — heavier than same-framework MFEs
Custom events for communication	Shell↔MFE uses DOM events, not React state or Angular `@Output`
Delayed first render	Script inject → Angular bootstrap → custom element definition adds ~100–300ms

Use Web Components when:

- The MFE uses a different framework than the Shell
- The MFE is a polyrepo or owned by an external team
- You want zero build-time coupling between host and remote
- You have fewer than ~20 MFEs

Do not use Web Components when:

- The MFE and Shell share the same framework — use Module Federation directly (lighter, faster)
- You need to pass complex objects or callbacks from Shell to MFE frequently
- The MFE needs deep URL routing integration with the Shell router
- You are building a platform with 50+ MFEs — single-spa's lifecycle management is better at that scale

---

Authentication — Single Sign-On Across All MFEs

Authentication is a shared concern. Users should log in once and every MFE should see their session — regardless of which framework it uses.

This is solved with a **shared auth singleton**:

1. Shell authenticates the user and stores the token in memory
2. The `@mfe/auth-lib` is loaded **once** and shared across all MFEs via the `shared` config
3. Every MFE calls `authLib.getToken()` — they all get the same token from the same store
4. When the token expires, the auth library silently refreshes it — all MFEs benefit automatically
5. Logging out clears the token everywhere — no MFE retains a stale session

The critical detail in Webpack Module Federation:

```
// webpack.config.js — Shell AND every MFE must declare auth-lib as a singleton
// This guarantees one instance is loaded, not one per MFE
shared: {
  '@mfe/auth-lib': { singleton: true, eager: true },
  react:           { singleton: true, requiredVersion: '^19.0.0' },
  'react-dom':     { singleton: true },
}
```

Why `singleton: true` matters: without this, each MFE would load its own copy of the auth library — each with its own token store. The user would need to log in once per MFE. `singleton` ensures only the first loaded copy is used; all others defer to it.

Design System — Consistency Without Coupling

All MFEs share the same visual language without being tied to each other:

- **Bootstrap 5 SCSS** is published as a shared package — all MFEs import it
- **Design tokens** (colours, spacing, typography) are CSS custom properties — change once, reflect everywhere on the next deploy
- **Web Components** like `<mfe-button>` and `<mfe-header>` are built once using native browser APIs — they work in every framework with no adapters

A designer changes the primary colour token. Every MFE updates on the next deploy — without touching a single MFE's code.

MFE Communication — How MFEs Talk to Each Other

Micro Frontends are deliberately isolated. This is the whole point — but it raises an important question: how does one MFE communicate with another, or with the Shell?

The answer depends on your integration strategy. Each approach uses a different channel, but the underlying principle is always the same: **communication must go through a shared interface**, never through a direct cross-MFE import.

The Three Channels

Channel	How it works	Typical use
Attributes / Props	Data passed at mount via HTML attributes, DOM properties, or JSX props	User ID, feature flags, token getter function
Events	Custom DOM events or a shared event bus	User actions, state changes, auth logout broadcast
Shared Singleton	A module loaded exactly once, used by all	Auth store, event bus, cart count

Communication by Strategy

Strategy	Shell → MFE	MFE → Shell	MFE ↔ MFE
Web Components	HTML attributes (strings) / DOM properties (objects)	Custom DOM event `bubbles: true`	`window.__mfeBus` EventTarget
single-spa	`customProps` at `registerApplication`	Shared singleton store	Shared store or window custom events
Module Federation	Direct JSX props (same framework) / DOM property (Angular WC)	Shared event bus singleton	`@mfe/event-bus` via `shared` config

What to Pass — and What Never to Pass

The communication contract is the public API of your MFE boundary.

Safe to pass: opaque IDs, role/permission flags, callback functions, feature flags, getter functions for tokens (not the token string itself), event signals with minimal payload.

Never pass: raw auth token strings in props or attributes (visible in browser DevTools), passwords or secrets, full user objects with PII (pass only what the MFE actually needs), entire Redux or NgRx state trees (large object on every render + tight coupling), DOM references or React/Angular framework internals.

> **The one rule across all strategies:** never import directly from another MFE's source code. All cross-MFE communication must go through a shared interface — props, events, or a shared library. This is what preserves independent deployability.

> For complete flow diagrams, code examples for all three strategies, security rules, performance guidelines, and event naming conventions — see: [08-mfe-communication.md]

How to Add a New Remote App

This is one of the strongest selling points of the architecture. Adding a new MFE to the platform takes less than 10 minutes of integration work.

Step 1 — Build and deploy the MFE

The remote team builds their app independently and deploys it anywhere — CDN, AWS S3, Azure Blob, Netlify. They expose one file: `remoteEntry.js`.

Step 2 — Add one entry to the manifest

```
"new-mfe": {
  "remoteEntry": "https://cdn.company.com/new-mfe/remoteEntry.js",
  "exposedModule": "./App",
  "framework": "react",
  "route": "/new-mfe",
  "active": true
}
```

Step 3 — Deploy the manifest

The manifest is a static JSON file. Deploying it takes seconds. No Shell code was changed. No Shell rebuild. No Shell redeployment.

Step 4 — Done

The Shell reads the updated manifest on next load. The new MFE appears automatically under its route. The Shell's generic loader handles navigation, route registration, and the home page card.

Using `active: false` as a feature flag:

Not ready to go live? Set `"active": false`. The MFE is registered but invisible. Flip it to `true` when ready — instant release, zero code change.

Technology Stack

Layer	Technology	Why
Shell	React 19 + Webpack 5	Smallest bundle, best MFE ecosystem
MFE integration	Webpack 5 Module Federation	Runtime loading — no code copying at build time
Angular MFEs	Angular 21.5 Zoneless + Standalone	No Zone.js interference, lighter bundle
CSS framework	Bootstrap 5 SCSS	No jQuery, design token support
Web Components	Native Custom Elements API	Works in every framework without adapters
Monorepo tooling	Nx Workspace	Build, test, dependency graph at scale
Orchestration (50+ apps)	single-spa	Native cross-framework lifecycle management
Authentication	JWT + OIDC-ready auth-lib	One token store shared by all MFEs

---

Angular 21.5 — A Special Mention

Angular's new **Zoneless** architecture (`provideZonelessChangeDetection()`) and **Standalone Components** make it a first-class citizen in this MFE platform:

- **No Zone.js** — eliminates the historical problem of Angular monkey-patching browser APIs and conflicting with React or Vue MFEs running alongside it in the same tab
- **No NgModule** — Angular MFEs now expose a clean bootstrap function, not an entire module tree. `createApplication()` creates a self-contained Angular instance with its own injector.
- **Signals** — fine-grained, predictable reactivity without Zone.js. Components update only when their specific signal changes — not on every async event in the page.

An Angular 21.5 MFE registers itself as a Web Component and integrates with the React Shell just as cleanly as a React MFE — with complete runtime isolation.

When to Use single-spa + Nx

For smaller setups (under 20 MFEs), a custom React Shell with Webpack Module Federation is sufficient and simpler to operate.

As the platform grows:

MFE Count	Recommended Setup
1 – 20	Custom React Shell + `mfe-manifest.json` (current setup)
20 – 50	Introduce Nx workspace for build tooling and boundary enforcement
50+	Add single-spa as orchestration layer + Nx import maps

single-spa replaces the custom Shell with a framework-agnostic Root Config. It manages the `bootstrap → mount → unmount` lifecycle for every MFE natively — React, Angular, Vue, and Vanilla — without any Web Component wrapping required.

Nx provides the monorepo tooling: generate new MFEs in seconds, run only affected builds in CI, visualise dependencies across 50+ projects, and enforce boundary rules so MFEs cannot accidentally import each other's internals.

Adding single-spa to an Nx Monorepo

> Full implementation details, code samples, and polyrepo configuration are covered in the dedicated guide: [07-single-spa-nx-implementation.md]

The high-level steps to integrate single-spa into the MFEDemo Nx workspace:

Step 1 — Install packages

```
npm install single-spa single-spa-react single-spa-angular import-map-overrides
```

Step 2 — Create the Root Config application

A minimal vanilla JS app that registers all MFEs. It reads the existing `mfe.manifest.json` — no new registry format needed.

```
nx generate @nx/web:application root-config --directory=apps/root-config
```

Step 3 — Set up SystemJS import maps

single-spa uses SystemJS import maps to resolve MFE URLs at runtime. An `importmap.json` maps each MFE name to its `remoteEntry` URL — the same URLs already in `mfe.manifest.json`.

Step 4 — Adapt each MFE to export single-spa lifecycles

Each MFE adds a framework adapter and exports three functions:

```
// The three functions single-spa calls on every MFE
export async function bootstrap(props) { /* one-time setup */ }
export async function mount(props)     { /* render the app */ }
export async function unmount(props)   { /* clean up — remove DOM, listeners, timers */ }
```

Why unmount matters: without it, Angular instances remain in memory after the user navigates away. single-spa's lifecycle model gives every MFE a clean way to tear itself down.

Step 5 — Configure the polyrepo `angularTodo`

- **Option A (recommended):** Keep the existing Web Component bootstrap. The Root Config wraps `<angular-todo-app>` with a thin single-spa lifecycle — **zero changes to the `angularTodo` repo**.
- **Option B (advanced):** Run `ng add single-spa-angular` inside the `angularTodo` polyrepo for full lifecycle management with proper Angular zone cleanup.

---

single-spa Pros and Cons

Advantages:

Advantage	Detail
True framework agnosticism	No Web Component wrapper needed — React, Angular, Vue use the same lifecycle protocol
Proper mount/unmount lifecycle	Memory is cleaned up when MFE unmounts — no Angular instances left running
No host framework dependency	Root Config is ~50 lines of vanilla JS — not tied to React
Battle-tested at scale	Used by companies with 100+ MFEs in production (Canopy Tax, Scania, IKEA)
DevTools built-in	`import-map-overrides` lets developers swap any MFE URL in the browser without rebuilding

Disadvantages:

Disadvantage	When it matters
Learning curve	`bootstrap/mount/unmount` + SystemJS + import maps is a new mental model
SystemJS is non-standard	Differs from native ESM — adds a runtime dependency to every page load
Per-MFE adapter required	Every MFE needs `single-spa-react`, `single-spa-angular`, or `single-spa-vue`
Shared dependency management	Shared packages must be in the import map and loaded only once — tricky to version

---

single-spa vs Web Component — Choosing for This Project

Scenario	Recommended approach
POC or fewer than 20 MFEs	Web Component + manifest (current setup — simpler, fewer deps)
20–50 MFEs, mixed frameworks	single-spa + Nx (lifecycle management becomes valuable)
50+ MFEs	single-spa + Nx import maps (full orchestration at scale)
Polyrepo external vendor MFE	Web Component (vendor only needs to publish a URL)
Polyrepo internal team MFE	Either — Web Component for simplicity, single-spa for memory control

For this POC with 4 MFEs, the Web Component + manifest approach is the right choice. single-spa is the natural next step when the platform grows past ~20 MFEs.

Workspace Tools for MFE — Beyond Nx

Nx is one of the most popular workspace tools for MFE, but it is not the only option. As of April 2026, there are several mature tools — each with different strengths.

Feature Comparison

Feature	Nx v21	Turborepo v2	pnpm Workspaces	Lerna v8	Rush v5	Bazel v8
MFE generators	Yes	No	No	No	No	No
Module Federation plugin	Built-in	Manual	Manual	Manual	Manual	Manual
Task caching	Local + Cloud	Local + Vercel (free)	None	Via Nx	Cobuild	Remote execution
Affected builds	`nx affected`	Content-aware hash	Manual scripting	Via Nx	Phased incremental	File-level
Visual dependency graph	Yes	No	No	No	No	CLI only
Module boundary rules	Yes (ESLint)	No	No	No	Approvals only	No
Angular first-class support	Yes	Manual	Works (basic)	Via Nx	Manual	Custom rules
Free remote caching	Paid (Nx Cloud)	Free (Vercel)	No	Via Nx	Self-hosted	Self-hosted
Learning curve	Medium	Low	Low	Low	High	Very High
Best MFE scale	20–200+ apps	5–50 apps	5–15 packages	Lib publishing	100–500+	1,000+ targets

Tool Summaries

Nx v21 — The richest MFE tooling available. Generates Shell and MFE projects, provides the Module Federation plugin, enforces module boundaries with ESLint, and visualises the full project dependency graph. The right default choice for any mixed-framework MFE platform.

Turborepo v2 — A minimal, Rust-powered task runner with zero opinions on structure. Extremely fast builds with content-aware caching. Free remote caching on Vercel. No MFE generators, no graph, no boundary enforcement — pairs well with pnpm Workspaces as a foundation layer.

pnpm Workspaces v10 — The strictest dependency isolation of any package manager. Fastest installs. Best used as the foundation layer under Nx or Turborepo in Hybrid setups, where local packages link across repo boundaries.

Lerna v8 — Powered by Nx internally. Its only unique value is the best-in-class versioning and npm publish workflow with automatic changelogs. Use it in Hybrid strategies where shared libs need to be published to npm on a versioned cadence.

Rush v5 — Microsoft's enterprise monorepo manager. Adds dependency approvals, enforced changelogs, and a shrinkwrap model designed for supply-chain security (PCI-DSS, SOC 2). High learning curve — justified only for organisations with strict governance requirements.

Bazel v8 — Google's polyglot build system. Hermetic, reproducible, distributed builds at enormous scale (10,000+ targets). Supports JS, Java, Go, Python in one build graph. The Starlark rule authoring is extremely complex — only justified when the build target count and language diversity genuinely require it.

Which Should You Pick?

- **Need MFE generators + Module Federation? → Nx**
- **Need fast caching only, no MFE opinions? → Turborepo**
- **Need to publish shared libs to npm? → Lerna** (Hybrid strategy)
- **Need only dependency isolation and fast installs? → pnpm Workspaces** as foundation
- **Enterprise governance, 500+ packages, compliance? → Rush**
- **Polyglot (JS + Java + Go) at massive scale? → Bazel**

Key Benefits — Summary

Benefit	How it is achieved
Independent deployments	Each MFE has its own CI/CD pipeline
Technology freedom	Any framework per MFE — Shell does not care
Shared design language	Shared SCSS tokens + Web Components
Single sign-on	Auth-lib singleton shared across all MFEs
Scale to 50+ apps	Add URL to manifest — zero Shell changes
Polyrepo support	Each MFE is its own Git repo
Team autonomy	Teams own their MFE end-to-end
Resilience	One MFE crash does not break others

Final Thought

Micro Frontends are not a silver bullet. For small applications with one team, they add unnecessary complexity. But for large products with multiple teams, multiple frameworks, and the need to deploy independently — they are the right architectural choice.

The platform described here is designed to start simple and grow into the single-spa + Nx model only when the scale demands it. **Every decision is reversible. Every MFE is isolated. Every team is autonomous.**

That is the promise of Micro Frontend Architecture.

Reference Resources

Resource	URL
MFEDemo GitHub (Shell + React + Vue + Angular)	https://github.com/sudeep31/MFEDemo
AngularTodo GitHub (Polyrepo Angular MFE)	https://github.com/sudeep31/angularTodo
single-spa + Nx — Full Implementation Guide	https://github.com/sudeep31/MFEDemo/blob/main/docs/07-single-spa-nx-implementation.md
MFE Communication — Complete Guide	./08-mfe-communication.md
Webpack 5 Module Federation docs	https://webpack.js.org/concepts/module-federation/
`@angular/elements` — Web Components	https://angular.dev/guide/elements
`@angular-architects/native-federation`	https://github.com/angular-architects/module-federation-plugin
single-spa official docs	https://single-spa.js.org/docs/getting-started-overview
Nx Module Federation plugin	https://nx.dev/technologies/module-federation/getting-started/intro
Angular Zoneless + Signals	https://angular.dev/guide/signals

Architecture designed for the MFEDemo POC — April 2026 Full technical documentation available in the project's `docs/` folder Source code: https://github.com/sudeep31/MFEDemo